Appendix

C LAB PROGRAMS

Learning Objectives

1. Write an 8085 Program to Swap two 8-bit Numbers.
2. a. Write a Program to Find the Largest of Two Numbers
   b. Write an 8085 Program to Find the Smallest of Two Numbers
3. Write an 8085 Program to find whether an 8-bit number is positive, negative or zero. If positive display EE, if negative display FF, if zero display DD.
4. Write an 8085 Program to check whether 4th bit of a number is zero or one. If 4th bit is 1 display FF, if 4th bit is 0 display DD.
5. Write an 8085 Program to Calculate the Sum of First Ten Natural Numbers.
6. Write an Assembly Language Program in 8085 Microprocessor to Find the Sum of Digits of an 8-bit Number.
7. Write an 8085 Program to Find the Reverse of an 8-bit Number
8. Write an 8085 Program to Check Whether 1-byte Number is a Palindrome or Not. If it is a Palindrome display FF otherwise display DD.
9. Write an 8085 Program to Check whether a Number is ODD or EVEN. If it is Even then Display DD, if is is Odd, then Display FF.
10. Write an 8085 Program to Count a Number of Ones in the given 8-bit Number.
11. Write an 8085 Program to Find Addition & Subtraction of two 8 –bit HEX Numbers.
12. Write an 8085 Program to Find Addition of two 16 –bit Numbers.
13. Write an 8085 Program to find Subtraction of two 16 –bit numbers.
14. Write an 8085 Program for Swapping of two 16-bit Numbers.
15. Write an 8085 Program to Implement 2 out of 5 codes.
16. Write an 8085 Program to Generate Fibonacci Series
17. Write an 8085 Program to Find the First Ten Terms of Odd and Even Numbers.
18. Write an 8085 Program to Find 4-Digit BCD Addition.
19. Write an 8085 Program to Find Multiplication of 2-digit BCD Numbers.
20. Write an 8085 Program to Find Division of two 8-bit Numbers.

Program 1 Write an 8085 Program to Swap two 8-bit Numbers.

Algorithm:

1. Load the value from memory location 2500H into Accumulator A.
2. Store the value of Accumulator A into register B.
3. Load the value from memory location 2501H into Accumulator A.
4. Store the value of Accumulator A into memory location 2500H.
5. Move the value from register B to Accumulator A.
6. Store the value of Accumulator A into memory location 2501H.
7. Halt the Program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the value from memory location 2500H into the Accumulator A.
MOV B, A	47	Move the value from Accumulator A to Register B to store it temporarily.
LDA 2501H	3A 01 25	Load the value from memory location 2501H into Accumulator A.
STA 2500H	32 00 25	Store the value from Accumulator A into memory location 2500H, effectively swapping the values.
MOV A, B	78	Move the value from Register B (original value of memory 2500H) back to Accumulator A.
STA 2501H	32 01 25	Store the value from Accumulator A into memory location 2501H (the second number location).
HLT	76	Halt the execution of the program.

Note:

- **If using GNU Sim 8085**, type the mnemonics code directly in the code editor, as there is no need to manually enter hex codes. During the assembly process, the simulator automatically converts the mnemonics into corresponding hex codes.
- **If using a Physical Microprocessor Kit**, the hex codes need to be entered manually in specific memory locations, such as storing 3A at 1000H, 00 at 1001H, 25 at 1002H, 47 at 1003H, and so on. After entering the hex codes, load the input data into memory locations such as 2500H and 2501H (which hold the values to be swapped). Then, update the Program Counter (PC) to start from memory location 1000H to execute the program.

Input & Output:

Input	
Address	**Data**
2500H	58 (3AH)
2501H	82 (52H)

Output	
Address	**Data**
2500H	82 (52H)
2501H	58 (3AH)

Note: In GNU Sim 8085, data is entered in decimal format, and the corresponding hexadecimal value is shown in brackets. For example, 58 in decimal is equivalent to 3AH in hexadecimal, and 82 in decimal is equivalent to 52H.

- **Input:** The memory location 2500H contains the value 58 (which is 3AH in hexadecimal), and memory location 2501H contains the value 82 (which is 52H in hexadecimal).
- **Output:** After executing the program, the values are swapped. Memory location 2500H now contains 82 (52H in hexadecimal), and memory location 2501H contains 58 (3AH in hexadecimal).

Program 2(a) **Write a Program to Find the Largest of Two Numbers**

Algorithm:

1. Load the first number from memory location 2500H into Accumulator A.
2. Move the value from Accumulator A into register B for temporary storage.
3. Load the second number from memory location 2501H into Accumulator A.
4. Compare the value in Accumulator A with the value in register B.
5. If the value in Accumulator A is greater than or equal to the value in register B, jump to the next step. Otherwise, move the value from register B back to Accumulator A.
6. Store the largest value from Accumulator A into memory location 2502H.
7. Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the first number from memory location 2500H into Accumulator A.
MOV B, A	47	Move the value in Accumulator A to register B for later comparison.
LDA 2501H	3A 01 25	Load the second number from memory location 2501H into Accumulator A.
CMP B	B8	Compare the value in Accumulator A with the value in register B. If A is greater or equal, the Carry Flag is not set (CY = 0).
JNC NEXT	D2 09 00	If Accumulator A is greater than or equal to the value in register B (CY = 0), jump to the label NEXT, as A already contains the largest value.
MOV A, B	78	If Accumulator A is smaller (CY = 1), load the value from register B back into Accumulator A (meaning B is the larger value).
NEXT: STA 2502H	32 02 25	Store the largest value from Accumulator A into memory location 2502H.
HLT	76	Halt the execution of the program.

Input & Output:

Input	
Address	**Data**
2500H	58 (3AH)
2501H	82 (52H)

Output	
Address	**Data**
2502H	82 (3AH)

- **Input:** Memory locations 2500H and 2501H contain 58 and 82 respectively.
- **Output:** The largest value (82) is stored in memory location 2502H.

Note : JNC NEXT" stands for "Jump if No Carry to NEXT". If the Carry Flag is not set (CY = 0), it means Accumulator A is greater than or equal to the value in register B. The program jumps to NEXT and skips loading a new value. If the Carry Flag is set (CY = 1), it means Accumulator A is smaller, so no jump occurs, and the value from register B (the larger value) is loaded into Accumulator A.

Program 2 (b) **Write a Program to Find the Smallest of Two Numbers**

Algorithm:

1. Load the first number from memory location 2500H into Accumulator A.
2. Move the value from Accumulator A into register B for temporary storage.
3. Load the second number from memory location 2501H into Accumulator A.
4. Compare the value in Accumulator A with the value in register B.
5. If the value in Accumulator A is less than or equal to the value in register B, jump to the next step. Otherwise, move the value from register B back to Accumulator A.
6. Store the smallest value from Accumulator A into memory location 2502H.
7. Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the first number from memory location 2500H into Accumulator A.
MOV B, A	47	Move the value in Accumulator A to register B for later comparison.
LDA 2501H	3A 01 25	Load the second number from memory location 2501H into Accumulator A.
CMP B	B8	Compare the value in Accumulator A with the value in register B. If A is less or equal, the Carry Flag is set (CY = 1).
JC NEXT	DA 09 00	If Accumulator A is less than or equal to the value in register B (CY = 1), jump to label NEXT, as A already contains the smallest value.
MOV A, B	78	If Accumulator A is greater (CY = 0), load the value from register B back into Accumulator A (meaning B is the smaller value).
NEXT: STA 2502H	32 02 25	Store the smallest value from Accumulator A into memory location 2502H.
HLT	76	Halt the execution of the program.

Input & Output:

Input	
Address	**Data**
2500H	58 (3AH)
2501H	82 (52H)

Output	
Address	**Data**
2502H	58 (3AH)

- **Input:** Memory locations 2500H and 2501H contain 58 and 82 respectively.
- **Output:** The smallest value (58) is stored in memory location 2502H.

Note : JC NEXT stands for "Jump if Carry to NEXT". If the Carry Flag is set (CY = 1), it means Accumulator A is less than or equal to the value in register B. The program jumps to NEXT and skips loading a new value. If the Carry Flag is not set (CY = 0), it means Accumulator A is greater, so no jump occurs, and the value from register B (the smaller value) is loaded into Accumulator A.

Program 3 **Write an 8085 Program to Find whether an 8-bit number is Positive, Negative or Zero.If positive display EE, if negative display FF, if zero display DD**

Algorithm:

1. Load the 8-bit number from memory location 2500H into Accumulator A.
2. Compare the value in Accumulator A with 00H.
3. If Accumulator A is equal to 00H, jump to the label ZERO and store 0DDH in memory location 2501H.
4. If the Sign Flag (S) is set (S = 1), jump to the label NEGATIVE and store 0FFH in memory location 2501H.
5. Otherwise, move 0EEH (positive value indicator) to Accumulator A and store it in memory location 2501H.
6. Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the 8-bit number from memory location 2500H into Accumulator A.
CPI 00H	FE 00	Compare the value in Accumulator A with 00H.
JZ ZERO	CA 12 00	This instruction checks if Accumulator A contains 00H (Z = 1). If Zero flag is set to 1, then the program jumps to the label ZERO.
JM NEGATIVE	FA 16 00	This instruction checks if the Sign Flag (S) is set (S = 1), which indicates that the number is negative. If S = 1, then jump to the label NEGATIVE.
MVI A, 0EEH	3E EE	Move the value 0EEH (indicating positive) into Accumulator A.
JMP STORE	C3 1A 00	Jump to STORE to save the result.
NEGATIVE: MVI A, 0FFH	3E FF	Move the value 0FFH (indicating negative) into Accumulator A.
JMP STORE	C3 1A 00	Jump to STORE to save the result.
ZERO: MVI A, 0DDH	3E DD	Move the value 0DDH (indicating zero) into Accumulator A.
STORE: STA 2501H	32 01 25	Store the result from Accumulator A into memory location 2501H.
HLT	76	Halt the program execution.

Depending on the flag status, the appropriate value (0EEH, 0FFH, or 0DDH) is loaded into Accumulator A and stored in memory location 2501H to indicate whether the original value was positive, negative, or zero.

Input & Output:

Input	
Address	**Data**
2500H	0 (00H)
2500H	78 (4EH)
2500H	128 (80H)

Output	
Address	**Data**
2501H	221 (DDH)
2501H	238 (EEH)
2501H	255 (FFH)

- **Input:** The 8-bit number in 2500H is checked to see if it is zero, positive, or negative.
  - 0 (00H): Represents zero.
  - 78 (4EH): A positive value.
  - 134 (86H): This value is treated as negative in the 8-bit representation.
- **Output:** After executing the program, the corresponding results are stored at memory location 2501H:
  - 221 (DDH): Indicates the input value(0) was zero.
  - 238 (EEH): Indicates the input value(78) was positive.
  - 255 (FFH): Indicates the input value(128) was negative.

Note:

- In 8-bit signed representation, numbers range from -128 to +127.
  - Positive values: Range from 0 to 127.
  - Negative values: Range from -128 to -1.
- The number 128 becomes negative when represented as an 8-bit value because the most significant bit (MSB) acts as a sign bit:
  - If the MSB is 0, the value is positive or zero.
  - If the MSB is 1, the value is negative.
- In this example, 134 (86H) has its MSB set to 1, indicating that it is a negative value in 8-bit representation.

Program 4 **Write an 8085 Program to Check whether 4th bit of a Number is Zero or One. If 4th bit is 1 display FF, if 4th bit is 0 display DD.**

Algorithm:

1. Load the 8-bit number from memory location 2500H into Accumulator A.
2. Perform AND operation with the value 08H to isolate the 4th bit.
3. If the result is zero, it means the 4th bit is 0. Jump to ZERO and store DDH in memory location 2501H.
4. If the result is non-zero, it means the 4th bit is 1. Store FFH in memory location 2501H.
5. Halt the program.

This algorithm isolates the 4th bit using bitmasking (08H). The result is stored based on whether the 4th bit is 0 or 1.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the number from memory location 2500H into Accumulator A.
ANI 08H	E6 08	Perform AND operation with 08H to isolate the 4th bit of the number.
JZ ZERO	CA 12 00	If the result is zero, jump to label ZERO (indicating 4th bit is 0).
MVI A, 0FFH	3E FF	Move the value 0FFH (indicating 4th bit is 1) into Accumulator A.
JMP STORE	C3 16 00	Jump to STORE to save the result.
ZERO: MVI A, DDH	3E DD	Move the value DDH (indicating 4th bit is 0) into Accumulator A.
STORE: STA 2501H	32 01 25	Store the result from Accumulator A into memory location 2501H.
HLT	76	Halt the program execution.

Input & Output:

Input	
Address	**Data**
2500H	6 (06H)
2500H	9 (09H)

Output	
Address	**Data**
2501H	221 (DDH)
2501H	255 (FFH)

- **Input:** Memory location 2500H contains values:
  - 6 (06H): The binary value of 6 is 0000 0110 and hence the 4th bit is 0
  - 9 (09H): The binary value of 9 is 0000 1001 and hence the 4th bit is 1.
- **Output:** Memory location 2501H stores the result:
  - 221 (DDH): Indicates that the 4th bit of the input value (6) is 0.
  - 255 (FFH): Indicates that the 4th bit of the input value (9) is 1.

Program 5 **Write an 8085 Program to Calculate the Sum of First Ten Natural Numbers.**

Algorithm:

1. Initialize Register C with 0AH (decimal 10), which represents the count of numbers to be added.
2. Initialize Register A with 00H (decimal 0) to store the running sum.
3. Initialize Register D with 01H (decimal 1) to keep track of the current number.
4. Add the current number (from Register D) to Accumulator A.
5. Increment the current number (Register D) by 1.
6. Decrement the counter (Register C) by 1.
7. If Register C is not zero, repeat the process of adding the next number.
8. Store the sum (in Accumulator A) into memory location 2500H.
9. Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
MVI C, 0AH	0E 0A	Load 0AH (count of 10) into Register C.
MVI A, 00H	3E 00	Load 00H into Accumulator A to initialize the sum to 0.
MVI D, 01H	16 01	Load 01H into Register D to start from the first natural number (1).
LOOP: ADD D	82	Add the value in Register D to Accumulator A.
INR D	14	Increment the value in Register D (next number).
DCR C	0D	Decrement the value in Register C by 1.
JNZ LOOP	C2 06 00	If Register C is not zero, jump to LOOP to continue adding.
STA 2500H	32 00 25	Store the final sum from Accumulator A into memory location 2500H
HLT	76	Halt the program execution.

1. **Initialization:**
   - Register C (0AH) keeps track of how many numbers are left to add (initially 10). Set to 10 (0AH in hexadecimal), indicating that there are 10 numbers to add (from 1 to 10).
   - Accumulator A (00H) starts with the value 0 to hold the running sum. Set to 0 (00H), so that the sum starts from zero. All subsequent numbers will be added to Accumulator A.
   - Register D (01H) holds the current natural number to be added. Set to 1 (01H), representing the first number in the sequence of natural numbers.
2. **Loop to Add the Numbers:**
   - The LOOP label starts the iteration for adding numbers. This loop label allow the microprocessor to repeat the instructions within this section.
   - ADD D adds the value in Register D (current number) to Accumulator A. The Accumulator is updated each time to maintain the running sum.
   - INR D increments the current number by 1.
   - DCR C decrements the counter to keep track of how many numbers have been added.
   - JNZ LOOP repeats the loop if the count is not zero to ensure all ten numbers are added.
3. **Store the Result:**
   - After the loop finishes, Accumulator A contains the sum (55 in decimal).
   - STA 2500H stores this sum in memory location 2500H.

Input & Output:

Output	
Address	**Data**
2500H	55 (37H)

- **Output:** The sum of the first ten natural numbers (1 + 2 + 3 + ... + 10 = 55) is stored in memory location 2500H. In hexadecimal, 55 is equivalent to 37H.

Program 6 **Write an Assembly Language Program in 8085 Microprocessor to Find the Sum of Digits of an 8-bit Number**

Algorithm:

1. Load the 8-bit number from memory location 2500H into Accumulator A.
2. Copy the value from Accumulator A into Register B for safekeeping.
3. Mask the higher nibble of Accumulator A to extract the units digit by performing an AND operation with 0FH (binary 0000 1111).
4. Move the units digit (currently in Accumulator A) to Register C to store it temporarily.
5. Restore the original value from Register B to Accumulator A.
6. Rotate Accumulator A left four times to bring the higher nibble (the tens digit) into the lower nibble position.
7. Mask the higher nibble again to extract the tens digit by performing an AND operation with 0FH.
8. Add the units digit (in Register C) to the Accumulator A.
9. Store the result from Accumulator A into memory location 2501H.
10. Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the 8-bit number from memory location 2500H into Accumulator A.
MOV B, A	47	Copy the value in Accumulator A to Register B for temporary storage.
ANI 0FH	E6 0F	Perform an AND operation to mask the higher nibble and retain only the lower nibble (units digit).
MOV C, A	4F	Move the value in Accumulator A (units digit) to Register C.
MOV A, B	78	Restore the original value from Register B to Accumulator A.
RLC	07	Rotate Accumulator A left once.
RLC	07	Rotate Accumulator A left again.
RLC	07	Rotate Accumulator A left again.
RLC	07	Rotate Accumulator A left one more time to bring the higher nibble into the lower nibble position.
ANI 0FH	E6 0F	Perform an AND operation to mask out the higher nibble, leaving only the tens digit in Accumulator A.
ADD C	81	Add the units digit (in Register C) to the tens digit in Accumulator A.
STA 2501H	32 01 25	Store the sum from Accumulator A into memory location 2501H
HLT	76	Halt the program execution.

This approach is a straightforward way of finding the sum of the individual decimal digits of an 8-bit number, using bitwise operations and rotations to isolate each digit.

Input & Output:

Input	
Address	**Data**
2500H	45 (2DH)

Output	
Address	**Data**
2501H	15 (2DH = 2 + D = 2 + 13 = 15)

1. Load 45 (2DH) into Accumulator A.
2. Copy 45 (2DH) to Register B for temporary storage
3. Mask the higher nibble:
   - 2DH in binary is 0010 1101.
   - ANI 0FH will result in 0000 1101 (i.e., 0DH), which is 13 in decimal.
4. Store the units digit (13) in Register C.
5. Restore Original Value (45 or 2DH) to Accumulator A.
6. Rotate Left Four Times: After four rotations, 2DH (0010 1101 in binary) becomes 1101 0010.
7. Mask to Extract Tens Digit: ANI 0FH results in 0000 0010 (i.e., 02H), which is 2 in decimal.
8. Add the Units Digit (2): The sum is 13 + 2 =15.
9. Store the Result (15) in memory location 2501H. The number provided in the memory location 2500H is 45 in decimal, which is represented as 2DH in hexadecimal.
   - Extract the units digit (D in hexadecimal, which is 13 in decimal).
   - Extract the tens digit (2 in hexadecimal, which is 2 in decimal).
   - The sum of 2 (tens digit) and 13 (units digit) should be 15 in decimal.

Program 7 **Write an 8085 Program to Find the Reverse of an 8-bit Number**

Algorithm:

1. Load the 8-bit number from memory location 2500H into Accumulator A.
2. Rotate Left the Accumulator A four times using the RLC instruction.
   - Each RLC operation rotates Accumulator A left through the Carry flag.
   - RLC shifts all bits in the Accumulator to the left. The leftmost bit (MSB) moves into the Carry flag, and the Carry flag is fed into the rightmost bit (LSB).
   - After four RLC instructions, the higher nibble (leftmost 4 bits) and lower nibble (rightmost 4 bits) effectively swap places.
4. Store the rotated value from Accumulator A into memory location 2501H.
5. Halt the program.

The given program rotates an 8-bit number left by four positions to effectively swap the higher and lower nibbles. For example, if the input is 4CH (binary 01001100), the output will be C4H (binary 11000100).

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the 8-bit number from memory location 2500H into Accumulator A.
RLC	07	Rotate Accumulator A left through Carry. The MSB moves into the Carry flag and the Carry flag moves into the LSB.
RLC	07	Rotate Accumulator A left again through Carry.
RLC	07	Rotate Accumulator A left for the third time through Carry.
RLC	07	Rotate Accumulator A left for the fourth time through Carry.
STA 2501H	32 01 25	Store the rotated value from Accumulator A into memory location 2501H.
HLT	76	Halt the program execution.

Input & Output:

Input	
Address	**Data**
2500H	49 (31H)

Output	
Address	**Data**
2501H	19 (13H)

1. Load 31H into Accumulator A (0011 0001).
2. Rotate Left Four Times:

   First Rotation (RLC): 0011 0001 becomes 0110 0010.

   Second Rotation (RLC): 0110 0010 becomes 1100 0100

   Third Rotation (RLC): 1100 0100 becomes 1000 1001

   Fourth Rotation (RLC): 1000 1001 becomes 0001 0011(in binary), which is 13H in hexadecimal.
3. Store the Result: The final rotated value (13H) is stored in memory location 2501H.

Program 8 **Write an 8085 Program to Check whether 1-byte Number is a Palindrome or Not. If it is a Palindrome display FF otherwise display DD.**

Algorithm:

1. Load the 8-bit number into Accumulator A.
2. Store the original value in Register B for later comparison.
3. Rotate Accumulator A Left four times to swap the higher nibble with the lower nibble.
4. Compare the original value (in Register B) with the current value in Accumulator A.
   - If equal, the number is a palindrome; otherwise, it is not.
5. Store FFH in memory location 2501H if it is a palindrome.
6. Store DDH in memory location 2501H if it is not a palindrome.
7. Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the 8-bit number from memory location 2500H into Accumulator A.
MOV B, A	47	Store the original value in Register B for later comparison.
RLC	07	Rotate Accumulator A left through the Carry flag.
RLC	07	Rotate Accumulator A left again through the Carry flag.
RLC	07	Rotate Accumulator A left again through the Carry flag.
RLC	07	Rotate Accumulator A left again through the Carry flag.
CMP B	B8	Compare the value in Accumulator A (rotated) with Register B (original).
JZ PALINDROME	CA 16 00	If Zero flag is set, jump to the label PALINDROME (indicating a palindrome).
MVI A, 0DDH	3E DD	Load DDH into Accumulator A (indicating it is not a palindrome).
JMP STORE	C3 1E 00	Jump to STORE to store the result.
PALINDROME: MVI A, 0FFH	3E FF	Load FFH into Accumulator A (indicating it is a palindrome).
STORE: STA 2501H	32 01 25	Store the result (FFH or DDH) in memory location 2501H
HLT	76	Halt the program execution.

Input & Output:

Input	
Address	**Data**
2500H	51 (33H)
2500H	23 (17H)

Output	
Address	**Data**
2501H	255 (FFH)
2501H	221 (DDH)

1. Load the value from memory location 2500H into Accumulator A (33H).
2. Store the original value (33H) in Register B for comparison.
3. Rotate Accumulator A left four times using the RLC instruction:
   Original binary: 00110011
   - First RLC: 01100110
   - Second RLC: 11001100
   - Third RLC: 10011001
   - Fourth RLC: 00110011 (which is 33H in hexadecimal).
4. Compare the original value (in Register B, which is 33H) with the rotated value (in Accumulator A, which is 33H).
5. Since the original value and rotated value are equal, the number is identified as a palindrome.

Program 9 **Write an 8085 Program to Check Whether a Number is ODD or EVEN. If Even no. display DD, if Odd no. display FF.**

Algorithm:

1. Load the 8-bit number from memory location 2500H into Accumulator A.
2. Check the Least Significant Bit (LSB): Use the AND operation with 01H to mask all bits except the LSB.
   - If the LSB is 0, the number is even.
   - If the LSB is 1, the number is odd.
3. Store the Result:
   - If the number is even, load DDH into the Accumulator and store it at memory location 2501H.
   - If the number is odd, load FFH into the Accumulator and store it at memory location 2501H.
4. Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the 8-bit number from memory location 2500H into Accumulator A.
ANI 01H	E6 01	Perform AND operation with 01H to check the LSB.
JZ EVEN	CA 0D 00	If the Zero flag is set (LSB is 0), jump to label EVEN.
MVI A, 0FFH	3E FF	Load FFH into Accumulator A (indicating it is odd).
JMP STORE	C3 11 00	Jump to STORE to store the result.
EVEN: MVI A, 0DDH	3E DD	Load DDH into Accumulator A (indicating it is even).
STORE: STA 2501H	32 01 25	Store the result (FFH or DDH) in memory location 2501H
HLT	76	Halt the program execution.

Input & Output:

Input	
Address	**Data**
2500H	21 (15H)
2500H	18 (12H)

Output	
Address	**Data**
2501H	255 (FFH)
2501H	221 (DDH)

1. Load 21H into Accumulator A.
2. Perform AND operation with 01H: 0010 0001 AND 00000001 = 00000001 (LSB is 0).
3. Zero flag is not set, indicating that the number is odd.
4. Load FFH into Accumulator A to indicate an odd number.
5. Store FFH in memory location 2501H.

Program 10 **Write an 8085 Program to count the Number of Ones in the given 8-bit Number.**

Algorithm:

1. Load the 8-bit number from memory location 2500H into Accumulator A.
2. Initialize the counter in Register B to 00H to keep track of the count of 1 bits.
3. Initialize a loop counter in Register C to 08H to iterate through all 8 bits.
4. Loop through each bit of the number:
   - Rotate Accumulator A right through Carry.
   - Check the Carry flag. If the Carry flag is set (1), increment the counter in Register B.
   - Decrement the loop counter and repeat until all bits are processed.
5. Store the count of 1 bits from Register B into memory location 2501H.
6. Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the 8-bit number from memory location 2500H into Accumulator A.
MVI B, 00H	06 00	Initialize Register B to 00H to keep track of the count of 1 bits.
MVI C, 08H	0E 08	Load 08H into Register C to set the loop for 8 bits.
LOOP: RAR	1F	Rotate Accumulator A right through Carry. The LSB moves into the Carry flag.
JNC SKIP	D2 0D 00	If the Carry flag is not set (0), jump to the label SKIP.
INR B	04	Increment Register B by 1 to count the 1 bit.
SKIP: DCR C	0D	Decrement Register C by 1 (to iterate through all 8 bits).
JNZ LOOP	C2 06 00	If Register C is not zero, jump back to LOOP to continue checking bits.
MOV A, B	78	Move the count of 1 bits from Register B to Accumulator A.
STA 2501H	32 01 25	Store the count of 1 bits in memory location 2501H.
HLT	76	Halt the program execution.

Input & Output:

Input Address	Input Data	Hexadecimal	Binary Representation	Count of 1 Bits	Output Address	Output Data
2500H	93	5DH	0101 1101	5	2501H	5
2500H	15	0FH	0000 1111	4	2501H	3

- **Input Data :** Two values are given: 93 (in decimal), which is 5DH in hexadecimal (01011101 in binary), and 15 (in decimal), which is 0FH in hexadecimal (00001111 in binary).
- **Output Data :** Memory Location 2501H stores the count of 1 bits found in each value, which are 5 for 5DH and 4 for 0FH.

1. **Original Value:** 0FH in hexadecimal is 00001111 in binary.
2. **Execution:**
   - Load 0FH (00001111 in binary) into Accumulator A.
   - Initialize Register B to 0 to keep track of the count of 1 bits.
   - Initialize Register C to 8 to set the loop counter to process all 8 bits.
3. **Loop to Count the Number of 1 Bits:**
   - First Rotation (RAR): The LSB is 1, so Carry flag is set. Increment Register B by 1. (Count=1)
   - Second Rotation: The LSB is 1, so Carry flag is set. Increment Register B by 1. (Count = 2)
   - Third Rotation: The LSB is 1, so Carry flag is set. Increment Register B by 1. (Count = 3)
   - Fourth Rotation: The LSB is 1, so Carry flag is set. Increment Register B by 1. (Count = 4)
   - Fifth to Eighth Rotations: All subsequent rotations result in 0 in the LSB, so Carry flag is not set, and Register B remains unchanged.
4. **Store the Count:**
   - Register B now holds the count of 1 bits, which is 4.
   - Store 4 in memory location 2501H.

Program 11 **Write an 8085 Program to Find the Addition and Subtraction of two 8-bit HEX Numbers.**

Algorithm:

1. Load the First Number:
   - Load the first 8-bit number from memory location 2500H into Accumulator A.
   - Move the value from Accumulator A to Register H for storage.
2. Load the Second Number:
   - Load the second 8-bit number from memory location 2501H into Accumulator A.
   - Move the value from Accumulator A to Register L for storage.
3. Perform Addition:
   - Move the first number from Register H to Accumulator A.
   - Add the value in Register L to Accumulator A.
   - If a carry is generated, set Register C to 01H. Otherwise, set Register C to 00H.
   - Store the sum in memory location 2502H.
   - Store the carry in memory location 2504H.
4. Perform Subtraction:
   - Move the first number from Register H to Accumulator A.
   - Subtract the value in Register L from Accumulator A.
   - Store the subtraction result in memory location 2503H.
5. Halt the Program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 25	Load the first 8-bit number from memory location 2500H into Accumulator A.
MOV H, A	67	Move the value from Accumulator A to Register H for storage.
LDA 2501H	3A 01 25	Load the second 8-bit number from memory location 2501H into Accumulator A.
MOV L, A	6F	Move the value from Accumulator A to Register L for storage.
MOV A, H	7C	Move the first number from Register H to Accumulator A.
ADD L	85	Add the value in Register L to Accumulator A.
JNC SKIP	D2 0F 00	If no carry is generated, jump to label SKIP.
MVI C, 01H	0E 01	If a carry is generated, set Register C to 01H.
JMP STORE1	C3 13 00	Jump to STORE1 to store the result.
SKIP: MVI C, 00H	0E 00	If no carry, set Register C to 00H.
STORE1: STA 2502H	32 02 25	Store the sum in memory location 2502H.
MOV A, C	79	Move the value of Register C (carry indicator) to Accumulator A.
STA 2504H	32 04 25	Store the carry value in memory location 2504H.
MOV A, H	7C	Move the first number from Register H to Accumulator A.
SUB L	95	Subtract the value in Register L from Accumulator A.
STA 2503H	32 03 25	Store the subtraction result in memory location 2503H.
HLT	76	Halt the program execution.

Input & Output:

Input	
Address	**Data**
2500H	124 (7CH)
2501H	46 (2EH)

Output		
Address	**Data**	**Description**
2502H	170 (AAH)	Sum of 7CH and 2EH.
2503H	78 (4EH)	Difference of 7CH and 2EH.
2504H	0	There was no carry generated during addition, so 00H is stored.

Program 12 **Write an 8085 Program to Find Addition of two 16 –bit Numbers.**

Algorithm:

1. Load the First 16-bit Number:
   - Load the first 16-bit number from memory locations 2500H (lower byte) and 2501H (higher byte) into the HL register pair.

2. Exchange Contents of HL and DE:
   - Exchange the contents of the HL register pair with the DE register pair to store the first 16-bit number in DE.
3. Load the Second 16-bit Number:
   - Load the second 16-bit number from memory locations 2502H (lower byte) and 2503H (higher byte) into the HL register pair.
4. Add the Lower Bytes:
   - Move the lower byte of the first number (in register E) into Accumulator A.
   - Add the lower byte of the second number (in register L) to Accumulator A.
   - Store the result in register L.
5. Add the Higher Bytes with Carry:
   - Move the higher byte of the first number (in register D) into Accumulator A.
   - Add the higher byte of the second number (in register H) along with the carry from the previous addition.
   - Store the result in register H.
6. Store the Final Result:
   - Store the 16-bit result (in registers H and L) into memory locations 2504H (lower byte) and 2505H (higher byte).
7. Halt the Program:
   - Stop the execution of the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LHLD 2500H	2A 00 25	Load the first 16-bit number from memory locations 2500H (lower byte) and 2501H (higher byte) into the HL register pair.
XCHG	EB	Exchange the contents of HL register pair with DE register pair to store the first number in DE.
LHLD 2502H	2A 02 25	Load the second 16-bit number from memory locations 2502H (lower byte) and 2503H (higher byte) into HL register pair.
MOV A, E	7B	Move the lower byte of the first number from register E into Accumulator A.
ADD L	85	Add the lower byte of the second number (in register L) to Accumulator A.
MOV L, A	6F	Store the result of the addition in register L.
MOV A, D	7A	Move the higher byte of the first number from register D into Accumulator A.
ADC H	8C	Add the higher byte of the second number (in register H) along with the carry from the previous addition.
MOV H, A	67	Store the result in register H.
SHLD 2504H	22 04 25	Store the 16-bit result from the HL register pair into memory locations 2504H (lower byte) and 2505H (higher byte).
HLT	76	Halt the execution of the program.

Input & Output:

Input		
Address	**Data**	**Description**
2500H	124 (7CH)	Lower byte of the first 16-bit number.
2501H	2 (02H)	Higher byte of the first 16-bit number.
2502H	46 (2EH)	Lower byte of the second 16-bit number.
2503H	1 (01H)	Higher byte of the second 16-bit number.

Output		
Address	**Data**	**Description**
2504H	170 (AAH)	Lower byte of the result (7CH + 2EH) (or) (124 + 46).
2505H	3 (03H)	Higher byte of the result (02H + 01H) or (2 + 1)

- **Inputs:**
  - The first 16-bit number is stored in memory locations 2500H (7CH) and 2501H (02H).
  - The second 16-bit number is stored in memory locations 2502H (2EH) and 2503H (01H).
  - These represent the numbers 027CH and 012EH in hexadecimal notation.
- **Output:**
  - After adding these two numbers, the result is 03AAH.
  - The lower byte of the result (AAH or 170 in decimal) is stored in memory location 2504H.
  - The higher byte of the result (03H or 3 in decimal) is stored in memory location 2505H.

Note : The carry generated during the addition of the lower bytes is properly handled during the higher byte addition using the ADC instruction.

Program 13 **Write an 8085 Program to Find Subtraction of two 16-bit Numbers.**

Algorithm:

1. Load the First 16-bit Number:
   - Load the first 16-bit number from memory locations 2500H (lower byte) and 2501H (higher byte) into the HL register pair.
2. Exchange Contents of HL and DE:
   - Exchange the contents of the HL register pair with the DE register pair to store the first 16-bit number in DE.
3. Load the Second 16-bit Number:
   - Load the second 16-bit number from memory locations 2502H (lower byte) and 2503H (higher byte) into the HL register pair.

4. Subtract the Lower Bytes:
   - Move the lower byte of the first number (in register E) into Accumulator A.
   - Subtract the lower byte of the second number (in register L) from Accumulator A.
   - Store the result in register L.
5. Subtract the Higher Bytes with Borrow:
   - Move the higher byte of the first number (in register D) into Accumulator A.
   - Subtract the higher byte of the second number (in register H) along with the borrow (if any) from the previous subtraction.
   - Store the result in register H.
6. Store the Final Result:
   - Store the 16-bit result (in registers H and L) into memory locations 2504H (lower byte) and 2505H (higher byte).
7. Halt the Program:
   - Stop the execution of the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LHLD 2500H	2A 00 25	Load the first 16-bit number from memory locations 2500H (lower byte) and 2501H (higher byte) into the HL register pair.
XCHG	EB	Exchange the contents of HL register pair with DE register pair to store the first number in DE.
LHLD 2502H	2A 02 25	Load the second 16-bit number from memory locations 2502H (lower byte) and 2503H (higher byte) into HL register pair.
MOV A, E	7B	Move the lower byte of the first number from register E into Accumulator A.
SUB L	95	Subtract the lower byte of the second number (in register L) from Accumulator A.
MOV L, A	6F	Store the result of the subtraction in register L.
MOV A, D	7A	Move the higher byte of the first number from register D into Accumulator A.
SBB H	9C	Subtract the higher byte of the second number (in register H) along with the borrow from the previous subtraction.
MOV H, A	67	Store the result in register H.
SHLD 2504H	22 04 25	Store the 16-bit result from the HL register pair into memory locations 2504H (lower byte) and 2505H (higher byte).
HLT	76	Halt the execution of the program.

Input & Output:

Input		
Address	**Data**	**Description**
2500H	124 (7CH)	Lower byte of the first 16-bit number.
2501H	2 (02H)	Higher byte of the first 16-bit number.
2502H	46 (2EH)	Lower byte of the second 16-bit number.
2503H	1 (01H)	Higher byte of the second 16-bit number.

Output		
Address	**Data**	**Description**
2504H	78 (4EH)	Lower byte of the result (7CH - 2EH) (or) (124 - 46).
2505H	1 (01H)	Higher byte of the result (02H - 01H) or (2 - 1)

- **Inputs:**
  - The first 16-bit number is stored in memory locations 2500H (7CH) and 2501H (02H).
  - The second 16-bit number is stored in memory locations 2502H (2EH) and 2503H (01H).
  - These represent the numbers 027CH and 012EH in hexadecimal notation.
- **Output:**
  - After subtracting these two numbers, the result is 014EH.
  - The lower byte of the result (4EH or 78 in decimal) is stored in memory location 2504H.
  - The higher byte of the result (01H or 1 in decimal) is stored in memory location 2505H.

Note : The borrow generated during the subtraction of the lower bytes is properly handled during the higher byte subtraction using the SBB instruction.

Program 14 **Write an 8085 Program for Swapping of two 16-bit Numbers.**

Algorithm:

1. Load the First 16-bit Number:
   - Load the first 16-bit number from memory locations 2500H and 2501H into the HL register pair.
2. Store the First 16-bit Number in Temporary Memory:
   - Store the 16-bit number (in HL) into temporary memory locations (3000H and 3001H).
3. Load the Second 16-bit Number:
   - Load the second 16-bit number from memory locations 2502H and 2503H into the HL register pair.
4. Store the Second 16-bit Number in the First Location:
   - Store the 16-bit number (in HL) into first number memory locations (2500H and 2501H).
5. Load the First Number from Temporary Memory:
   - Load the first number from temporary memory locations (3000H and 3001H) into the HL register pair.
6. Store the First Number in the Second Location:
   - Store the 16-bit number (in HL) into second number memory locations (2502H and 2503H).
7. Halt the Program:
   - Stop the execution of the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LHLD 2500H	2A 00 25	Load the first 16-bit number from memory locations 2500H (lower byte) and 2501H (higher byte) into HL register pair.
SHLD 3000H	22 00 30	Store the first 16-bit number from HL register pair into temporary memory locations 3000H and 3001H.
LHLD 2502H	2A 02 25	Load the second 16-bit number from memory locations 2502H (lower byte) and 2503H (higher byte) into HL register pair.
SHLD 2500H	22 00 25	Store the second 16-bit number from HL register pair into first number memory locations 2500H and 2501H.
LHLD 3000H	2A 00 30	Load the first 16-bit number from temporary memory locations 3000H and 3001H into HL register pair.
SHLD 2502H	22 02 25	Store the first 16-bit number from HL register pair into second number memory locations 2502H and 2503H.
HLT	76	Halt the execution of the program.

Input & Output:

Input		
Address	**Data**	**Description**
2500H	124 (7CH)	Lower byte of the first 16-bit number.
2501H	2 (02H)	Higher byte of the first 16-bit number.
2502H	46 (2EH)	Lower byte of the second 16-bit number.
2503H	1 (01H)	Higher byte of the second 16-bit number.

Output		
Address	**Data**	**Description**
2500H	46 (2EH)	Lower byte of the second 16-bit number.
2501H	1 (01H)	Higher byte of the second 16-bit number.
2502H	124 (7CH)	Lower byte of the first 16-bit number.
2503H	2 (02H)	Higher byte of the first 16-bit number.

- **Inputs:**
  - The first 16-bit number is stored in memory locations 2500H and 2501H (which are 45H and 12H respectively).
  - The second 16-bit number is stored in memory locations 2502H and 2503H (which are 78H and 34H respectively).
- **Outputs:**
  - The numbers are swapped after execution.
  - The first 16-bit number (45H 12H) now replaces the second 16-bit number (2502H and 2503H).
  - The second 16-bit number (78H 34H) now replaces the first 16-bit number (2500H and

Program 15 **Write an 8085 Program to Implement 2 out of 5 Codes. Display FF for Valid Code and DD for Invalid Code.**

What is 2 out of 5 Code?

2 out of 5 code is a 5-bit binary code used for error detection and representation, particularly where data reliability is crucial. The characteristic of this code is that exactly two of the five bits are set to '1', and the remaining three bits are set to '0'.

Examples :

- Valid 2 out of 5 Code: 00110, 01001, 10010
- Invalid Code: 00001, 11100 (these do not have exactly two 1s).

Representation of 5-Bit Numbers in 8085

- A 5-bit number can take values from 0 to 31 (in decimal), or 00000 to 11111 in binary.
- To store or manipulate a 5-bit value in 8085, we need to use 8-bit registers. Therefore, we will need to ensure that the upper 3 bits are 0 to represent the 5-bit value in an 8-bit format.
- For example: If we have a 5-bit value like 00011 (which is 3 in decimal), we would load it into an 8-bit register as 00000011.
- Use the 5-bit value and convert it into its equivalent 8-bit representation, ensuring the upper 3 bits are 0.

The following 8085 assembly program verifies if the value from a memory location is a valid 2 out of 5 code. If the value is valid, it stores FFH in another memory location; if it is invalid, it stores DDH.

Algorithm:

1. Load the Value:
   - Load the 5-bit number from memory location 2500H into the Accumulator A.
   - Mask the upper 3 bits using an AND operation (ANI 1FH) to ensure only 5 bits are considered.
2. Initialize Counters:
   - Set up a loop counter (C) to count through all 5 bits.
   - Initialize Register D to 0 to count the number of 1 bits.
3. Loop to Count 1 Bits:
   - Rotate Accumulator A right through the carry flag (RRC).
   - If the carry flag is set (indicating a 1 bit), increment Register D.
   - Decrement the loop counter (C) and repeat until all 5 bits are checked.
4. Compare the Count:
   - Load value 2 into the Accumulator A and compare with Register D.
   - If the number of 1s is not equal to 2, mark the value as invalid (DDH).
5. Store the Result:
   - If valid, load FFH to indicate validity.
   - Store the result (FFH or DDH) in memory location 2501H.
6. Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LDA 2500H	3A 00 2	Load the value from memory location 2500H into Accumulator A.
ANI 1FH	E6 1F	Perform an AND operation with 1FH (00011111 in binary) to mask the upper 3 bits, leaving only the lower 5 bits.
MVI C, 05H	0E 05	Move the value 05H into Register C to set up a loop counter to 5.
MVI D, 00H	16 00	Move the value 00H into Register D to use as a counter for counting 1 bits.
LOOP: RRC	0F	Rotate the value in Accumulator A right through the carry flag. The LSB goes into the carry flag, and all bits are shifted right.
JNC SKIP	D2 0C 00	Jump to SKIP if the carry flag is not set (CY = 0). This indicates that the current bit is 0.
INR D	14	Increment the value in Register D to count the 1 bit if the carry flag is set (CY = 1).
SKIP: DCR C	0D	Decrement the value in Register C (loop counter).
JNZ LOOP	C2 08 00	Jump to LOOP if the value in Register C is not zero (C ≠ 0). This ensures all 5 bits are checked.
MVI A, 02H	3E 02	Load the value 02H into Accumulator A. This is used to compare the number of 1 bits.
CMP D	B9	Compare the value in Accumulator A with the value in Register D.
JNZ INVALID	C2 14 00	Jump to INVALID if the value in Register D is not equal to 2 (A ≠ D).
MVI A, 0FFH	3E FF	If the value is valid (count of 1s is 2), load FFH into Accumulator A to indicate validity.
JMP STORE	C3 19 00	Jump to the STORE label to store the result.
INVALID: MVI A, 0DDH	3E DD	Load DDH into Accumulator A to indicate invalidity.
STORE: STA 2501H	32 01 25	Store the value from Accumulator A into memory location 2501H.
HLT	76	Halt the program execution.

Input & Output:

Input	
Address	**Data**
2500H	9 (09H) - 0000 1010
2500H	16 (10H) - 0001 0000

Output	
Address	**Data**
2501H	255 (FFH) - Valid Code
2501H	221 (DDH) - Invalid Code

- **For input value 9 (09H):**
  - The 5-bit value (01001) contains two '1' bits, which makes it a valid 2 out of 5 code.
  - Therefore, FFH (255 in decimal) is stored at memory location 2501H.
- **For input value 16 (10H):**
  - The 5-bit value (10000) contains only one '1' bit, making it an invalid 2 out of 5 code.
  - Hence, DDH (221 in decimal) is stored at memory location 2501H.

Program 16 **Write an 8085 Program to Generate Fibonacci Series**

The Fibonacci series is a sequence of numbers in which each number after the first two is the sum of the two preceding ones. The sequence typically starts with 0 and 1. The series progresses as follows: 0, 1, 1, 2, 3, 5, 8, 13, ... Each new number is obtained by adding the two previous numbers.

Algorithm:

1. Initialization:
   - Load the count of numbers to be generated (example, 10) into Register D using MVI D, 0AH.
   - Load the starting address (2500H) into the HL register pair using LXI H, 2500H.
   - Set Register B to 0 (first number in the Fibonacci series) and Register C to 1 (second number in the series).
2. Store First Two Terms:
   - Store the value 00H (first Fibonacci number) in memory location 2500H using MOV M, B.
   - Increment the memory pointer (H-L) to point to the next memory address using INX H.
   - Store the value 01H (second Fibonacci number) in memory location 2501H using MOV M, C.
   - Increment the memory pointer (H-L) to point to the next memory address using INX H.
   - Decrement the term counter (D) by 1 using DCR D, since two terms have already been stored.
3. Generate Fibonacci Terms Using Loop:
   - Start Loop:
     - Move the value from Register B into Accumulator A using MOV A, B.
     - Add the value in Register C to Accumulator A using ADD C. The result is stored in Accumulator A (next Fibonacci term).
     - Store the result in memory at the address pointed to by H-L using MOV M, A.
     - Increment the memory pointer (H-L) using INX H to point to the next address.
   - Update Previous Terms:
     - Move the value in Register C to Register B using MOV B, C (the previous term becomes the second last term).
     - Move the value in Accumulator A to Register C using MOV C, A (the newly calculated term becomes the last term).

- Decrement and Loop:
  - Decrement the term counter (D) using DCR D.
  - If D is not zero, jump back to LOOP using JNZ LOOP to continue generating the Fibonacci series.

4. Store Final Result:
   - Once all terms are generated and stored in memory, halt the program using HLT.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
MVI D, 0AH	16 0A	Load Register D with 10 (number of terms to generate).
LXI H, 2500H	21 00 25	Load HL pair with starting address 2500H to store the Fibonacci numbers.
MVI B, 00H	06 00	Set Register B to 0 (first term of Fibonacci series).
MVI C, 01H	0E 01	Set Register C to 1 (second term of Fibonacci series).
MOV M, B	77	Store the value in Register B (0) at the address pointed to by HL (2500H).
INX H	23	Increment HL to point to the next address (2501H).
MOV M, C	71	Store the value in Register C (1) at the address pointed to by HL (2501H).
INX H	23	Increment HL to point to the next address (2502H).
DCR D	15	Decrement the term counter (D) since two terms are already stored.
LOOP: MOV A, B	78	Move the value from Register B into Accumulator A.
ADD C	81	Add the value in Register C to Accumulator A to get the next Fibonacci term.
MOV M, A	77	Store the new term in memory pointed by HL.
INX H	23	Increment HL to point to the next address for storing the next term.
MOV B, C	41	Move the value in Register C to Register B (update to second last term).
MOV C, A	4F	Move the new term from Accumulator A to Register C (update to last term).
DCR D	15	Decrement the term counter.
JNZ LOOP	C2 08 00	Jump back to LOOP if D is not zero.
HLT	76	Halt the program after generating all terms.

Input & Output:

Output	
Address	**Data**
2501H	0 (00H)
2501H	1 (01H)
2502H	1 (01H)
2503H	2 (02H)
2504H	3 (03H)
2505H	5 (05H)
2506H	8 (08H)
2507H	13 (0DH)
2508H	21 (15H)
2509H	34 (22H)

- The program generates 10 numbers of the Fibonacci series, starting from 0 and 1.
- The generated sequence (0, 1, 1, 2, 3, 5, 8, 13, 21, 34) is stored in memory locations from 2500H to 2509H.

Program 17 **Write an 8085 Program to Find the Sum of First Ten terms of odd and even numbers.**

The program is to calculate the sum of the first ten terms of odd numbers and the sum of the first ten terms of even numbers. The program will store the results in memory locations, with the sum of even numbers and sum of odd numbers stored in two separate locations.

- Sum of the first ten even numbers (2, 4, 6, ..., 20).
- Sum of the first ten odd numbers (1, 3, 5, ..., 19).

Algorithm:

1. Initialization:
   - Load the counter (D) with 10 to represent the number of terms.
   - Load Register H-L with the starting memory address (2500H) where the result of even and odd sums will be stored.
2. Calculate Sum of First 10 Even Numbers:
   - Load Register B with the value 2 (the first even number).
   - Initialize Register C to 0 to accumulate the sum of even numbers.
   - Clear Accumulator A to start summing.
   - Repeat the following steps 10 times:
     - Add Register B (current even number) to Accumulator A.
     - Store the result of the addition into Register C.
     - Increment Register B by 2 to move to the next even number.
     - Decrement Register D after each addition until D = 0.

- Store the sum of the even numbers in memory location 2500H.

3. Reset for Sum of Odd Numbers:
   - Reload Register D with 10.
   - Load Register B with the value 1 (the first odd number).
   - Initialize Register C to 0 to accumulate the sum of odd numbers.
   - Clear Accumulator A again for summing.
4. Calculate Sum of First 10 Odd Numbers:
   - Repeat the following steps 10 times:
     - Add Register B (current odd number) to Accumulator A.
     - Store the result of the addition into Register C.
     - Increment Register B by 2 to move to the next odd number.
     - Decrement Register D after each addition until D = 0.
   - Store the sum of the odd numbers in memory location 2501H.
5. End the Program:
   - Halt the program execution.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
MVI D, 0AH	16 0A	Load Register D with 10 (number of terms to calculate).
MVI B, 02H	06 02	Load Register B with the first even number (2).
MVI C, 00H	0E 00	Initialize Register C to 0 to hold the sum of even numbers.
MVI A, 00H	3E 00	Clear Accumulator A to start adding even numbers.
SUM_EVEN: ADD B	80	Add the value in Register B (current even number) to Accumulator A.
MOV C, A	4F	Store the result of the addition in Register C.
INR B	04	Increment Register B to get the next even number.
INR B	04	Increment again ('B' now holds the next even number).
DCR D	15	Decrement Register D (to count ten terms).
JNZ SUM_EVEN	C2 08 00	If D is not zero, jump to SUM_EVEN.
LXI H, 2500H	21 00 25	Load H-L pair with the starting memory address (2500H).
MOV M, C	77	Store the sum of even numbers in memory location 2500H.
MVI D, 0AH	16 0A	Reload Register D with 10 for odd number calculation.
MVI B, 01H	06 01	Load Register B with the first odd number (1).
MVI C, 00H	0E 00	Initialize Register C to 0 to hold the sum of odd numbers.
MVI A, 00H	3E 00	Clear Accumulator A for summing odd numbers.
SUM_ODD: ADD B	80	Add the value in Register B (current odd number) to Accumulator A.

MOV C, A	4F	Store the result of the addition in Register C.
INR B	04	Increment Register B to get the next odd number.
INR B	04	Increment again (`B` now holds the next odd number).
DCR D	15	Decrement Register D (to count ten terms).
JNZ SUM_ODD	C2 14 00	If D is not zero, jump to SUM_ODD.
LXI H, 2501H	21 01 25	Load H-L pair with memory address 2501H.
MOV M, C	77	Store the sum of odd numbers in memory location 2501H.
HLT	76	Halt the program execution.

Input & Output:

Output		
Address	**Data**	**Description**
2500H	110 (6EH)	Sum of the first 10 even numbers (2 + 4 + 6 + 8 + ... + 20 = 110).
2501H	100 (64H)	Sum of the first 10 odd numbers (1 + 3 + 5 + 7 + ... + 19 = 100).

Program 18 **Write an 8085 Program to Find 4-Digit BCD Addition.**

The goal is to add two 16-bit BCD (Binary Coded Decimal) numbers using the 8085 microprocessor. The two 16-bit numbers are split into two parts each: lower byte and higher byte, with their respective values stored in memory locations. The lower and higher bytes are to be added individually, with proper carry handling and BCD adjustment using DAA to ensure valid BCD results.

The program performs:

- Addition of lower bytes from the two numbers.
- Adjustment to handle any generated carry.
- Addition of higher bytes and incorporation of carry from the lower bytes.
- Storing the lower byte, higher byte, and carry result into memory.

Algorithm:

1. Initialize Carry Register:
   - Register C is used to store the carry flag.
   - Set C = 00H initially.
2. Load the First 16-bit Number into HL Register Pair:
   - Load the value from memory locations 2500H and 2501H into HL.
3. Store First Number in DE Register Pair:
   - Use XCHG to exchange HL and DE to store the first number in the DE register pair.
4. Load the Second 16-bit Number into HL Register Pair:
   - Load the value from memory locations 2502H and 2503H into HL.
5. Add Lower Bytes:
   - Move the lower byte of the first number (in E) into Accumulator A.
   - Add the lower byte of the second number (in L) to Accumulator A.

- Adjust the result with DAA to ensure a valid BCD result.
- If carry is generated, increment the D register (higher byte of DE).
- Set Register C = 01H if a carry occurs.
- If there is no carry, set C = 00H.
- Store the result into memory location 2504H.

6. Add Higher Bytes:
   - Move the higher byte of the first number (in D) to Accumulator A.
   - Add the higher byte of the second number (in H) along with the carry.
   - Use DAA to adjust the result to ensure a valid BCD result.
   - Store the higher byte result in memory location 2505H.
7. Store Carry:
   - Store the value in Register C in memory location 2506H.
8. End the Program:
   - Halt the program execution with HLT.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
MVI C, 00H	0E 00	Initialize Register C to store carry.
LHLD 2500H	2A 00 25	Load the first 16-bit number from 2500H and 2501H into HL.
XCHG	EB	Exchange HL and DE, storing the first number in DE.
LHLD 2502H	2A 02 25	Load the second 16-bit number from 2502H and 2503H into HL.
MOV A, E	7B	Move the lower byte of the first number (from E) to Accumulator A.
ADD L	85	Add the lower byte of the second number (from L) to Accumulator A.
DAA	27	Adjust the result to ensure valid BCD format.
JNC NO_CARRY1	D2 xx xx	If no carry, jump to NO_CARRY1. (Address to be calculated during assembly).
INR D	14	Increment Register D (higher byte of DE) if there was a carry.
MVI C, 01H	0E 01	Set Register C to 01H if a carry is generated.
JMP STORE_LOWER	C3 xx xx	Jump to STORE_LOWER label. (Address to be calculated during assembly).
NO_CARRY1	XX	(Label address, no hex code)
MVI C, 00H	0E 00	Set Register C to 00H if no carry.
STORE_LOWER	XX	(Label address, no hex code)
STA 2504H	32 04 25	Store the result of the lower byte addition in memory location 2504H.

MOV A, D	7A	Move the higher byte of the first number (from D) to Accumulator A.
ADI 00H	C6 00	Add 00H to Accumulator A to update carry status if needed.
ADC H	8C	Add the higher byte of the second number (in H) along with any carry.
DAA	27	Adjust the result to ensure a valid BCD format.
STA 2505H	32 05 25	Store the higher byte result in memory location 2505H.
MOV A, C	79	Move the carry indicator from Register C to Accumulator A.
STA 2506H	32 06 25	Store the carry value in memory location 2506H.
HLT	76	Halt the program.

Input & Output:

Input		
Address	**Data**	**Description**
2500H	56	Lower byte of first number (DE).
2501H	34	Higher byte of first number (DE).
2502H	90	Lower byte of second number (HL).
2503H	78	Higher byte of second number (HL).

Output		
Address	**Data**	**Description**
2504H	46	Lower byte sum of BCD addition.
2505H	13	Higher byte sum of BCD addition.
2506H	01	Carry (third byte) of BCD result.

- Lower Byte Result (2504H):
  - Adding 56 (E) + 90 (L) gives 146.
  - After BCD adjustment (DAA), the valid BCD value is 46.
- Higher Byte Result (2505H):
  - Adding 34 (D) + 78 (H) + 1 (carry) gives 113.
  - After BCD adjustment, the value 13 is stored.
- Carry Value (2506H):
  - The carry from the previous addition is 01 (meaning an overflow occurred), which is stored in 2506H.

Note : The DAA instruction might not function correctly in GNU SIM 8085 due to limitations in the simulator's implementation of BCD adjustment. This program has been tested successfully in other online simulators, such as https://www.sim8085.com/, where BCD addition and carry adjustments work as expected.

Program 19 **Write an 8085 Program to Find Multiplication of 2-digit BCD Numbers.**

Algorithm:

1. Initialization:
   - Load 07H into register C.
   - Load 05H into register D.
   - Clear the Accumulator A to initialize it to 0.
   - Set register B to 0 to store any carries.
2. Loop for Repeated Addition:
   - Loop Start:
     - Add the value in register C to Accumulator A.
     - Adjust the Accumulator A using DAA to ensure the result is valid BCD.
     - If no carry is generated, skip the increment of B.
     - If a carry is generated, increment register B.
   - Decrement register D.
   - If D is not zero, repeat the loop.
3. Store the Result:
   - Store the final result from Accumulator A into memory location 2500H.
   - Store the carry count (if any) from register B into memory location 2501H.
4. End Program:
   - Halt the program.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
MVI C, 07H	0E 07	Load immediate data 07H into register C.
MVI D, 05H	16 05	Load immediate data 05H into register D.
XRA A	AF	Clear Accumulator A to 0
MOV B, A	47	Move the content of Accumulator A to register B (initialize B to 0).
Loop: ADD C	81	Add the value in register C to Accumulator A.
DAA	27	Decimal Adjust Accumulator A to ensure a valid BCD result.
JNC NEXT	D2 1E 00	If no carry is generated, jump to NEXT.
INR B	04	Increment register B if a carry is generated.
NEXT: DCR D	15	Decrement register D.
JNZ Loop	C2 12 00	If D is not zero, jump to the Loop for repeated addition.
STA 2500H	32 00 25	Store the content of Accumulator A into memory location 2500H.
MOV A, B	78	Move the content of register B to Accumulator A.
STA 2501H	32 01 25	Store the value of Accumulator A into memory location 2501H.
HLT	76	Halt the program execution.

Input & Output:

Output		
Address	**Data**	**Description**
2500H	35H	The product of 07H and 05H (BCD format).
2501H	00H	Carry generated during addition (if any)

Program 20 **Write 8085 Program to Divide two 8-bit Numbers**

This program performs the division of two 8-bit numbers in an 8085 microprocessor. The dividend and divisor are taken from memory locations, and the quotient and remainder are stored in different memory locations.

Algorithm:

1. Load Dividend and Divisor:
   - Load the address of the dividend (2500H) into the HL register pair.
   - Load the dividend from memory location 2500H into register B.
   - Increment HL to point to the divisor at memory location 2501H.
   - Load the divisor from memory location 2501H into Accumulator A and move it to Register D for comparison.
2. Initialize Quotient and Remainder:
   - Set the quotient (C) to 0.
   - Move the dividend from B to Accumulator A.
3. Division Loop:
   - Compare the current remainder (in Accumulator A) with the divisor (in D).
   - If the remainder is less than the divisor, jump to the DONE label.
   - Otherwise, subtract the divisor from the remainder and increment the quotient (C).
   - Repeat until the remainder is less than the divisor.
4. Store Results:
   - Store the quotient in memory location 2502H.
   - Store the remainder in memory location 2503H.
5. Halt the Program:
   - Halt the program to indicate completion.

Mnemonic Code with Hex Code and Explanation

Mnemonic Code	Hex Code	Explanation
LXI H, 2500H	21 00 25	Load address 2500H into HL register pair.
MOV B, M	46	Load dividend from memory location 2500H into B (dividend).
INX H	23	Increment HL to point to memory location 2501H.

MOV A, M	7E	Load divisor from memory location 2501H into Accumulator A.
MOV D, A	57	Copy divisor from A to D.
MVI C, 00H	0E 00	Set quotient (C) to 0.
MOV A, B	78	Move dividend (from B) to A.
DIV_LOOP: CMP D	BA	Compare A with D (divisor).
JC DONE	DA XX XX	Jump to DONE if A is less than D.
SUB D	92	Subtract D from A.
INR C	0C	Increment quotient C by 1.
JMP DIV_LOOP	C3 XX XX	Jump to DIV_LOOP to continue division.
DONE: MOV B, A	47	Move remainder from A to B.
LXI H, 2502H	21 02 25	Load address 2502H to store quotient and remainder.
MOV M, C	70	Store quotient in memory location 2502H.
INX H	23	Increment HL to point to 2503H.
MOV M, B	70	Store remainder in memory location 2503H.
HLT	76	Halt the program execution.

Input & Output:

Input		
Address	**Data**	**Description**
2500H	80 (50H)	Dividend
2501H	03 (03H)	Divisor

Output		
Address	**Data**	**Description**
2502H	26 (1AH)	Quotient
2503H	02 (02H)	Remainder

- The dividend (80) is stored in memory location 2500H and loaded into register B.
- The divisor (3) is stored in memory location 2501H and loaded into Accumulator A and Register D.
- The program uses repeated subtraction to divide the dividend by the divisor until the remainder is less than the divisor.
  - For each subtraction, the quotient register (C) is incremented.
- The quotient is stored in memory location 2502H and the remainder in memory location 2503H.

●●●●